

Universidade Tecnológica Federal do Paraná

Programa de Pós-Graduação em Ciência da Computação

PPGCC02 – Estrutura de Dados (001 CM)

Trabalho Prático: Benchmarking e Análise Empírica de Algoritmos de Ordenação

Alunos: Gabriel Pereira do Carmo
Pedro Acácio Rodrigues

Professor: André Luiz Satoshi Kawamoto

Campo Mourão
Julho de 2026

Sumário

1	Resumo	1
2	Apresentação	1
3	Repositório e Reprodutibilidade	2
4	Seção dos Testes e Ambiente de Execução	2
4.1	Ambiente utilizado por Gabriel Pereira do Carmo	2
4.2	Ambiente utilizado por Pedro Acácio Rodrigues	3
5	Descrição de Atividades	3
5.1	Problema 1	4
5.2	Problema 2	4
5.3	Problema 3	4
6	Análise dos Resultados	5
6.1	Problema 1: Operações Teóricas vs. Tempo Real	5
6.2	Problema 2: Quick Sort Clássico em Vetor Ordenado	5
6.3	Problema 3: Dados Quase Ordenados e Shell Sort	6
7	Conclusão	7

1 Resumo

Este relatório apresenta uma análise empírica de algoritmos clássicos de ordenação implementados em linguagem C. A comparação foi realizada por meio da coleta de tempo de execução, comparações e trocas ou movimentações no vetor.

Os resultados mostram que o desempenho dos algoritmos depende fortemente do perfil da entrada. Algoritmos simples, como Insertion Sort, podem ser muito eficientes em vetores já ordenados ou quase ordenados, enquanto estratégias como o Quick Sort clássico podem se degradar quando a escolha do pivô é desfavorável.

2 Apresentação

O trabalho prático propõe a implementação, instrumentação e comparação empírica de seis algoritmos de ordenação: Bubble Sort, Insertion Sort, Selection Sort, Shell Sort, Quick Sort e Heap Sort. O código-fonte e os dados gerados estão disponíveis no repositório do projeto.

O objetivo é observar, por meio de medições empíricas, como esses algoritmos se comportam em cenários distintos de entrada. Além da complexidade assintótica conhecida, foram coletadas métricas concretas de execução, permitindo comparar o custo teórico dos algoritmos com o comportamento real observado durante os testes.

Todos os algoritmos foram implementados em C e instrumentados por meio da estrutura `Metricas`, que registra:

- tempo de execução em segundos;
- número de comparações entre chaves;
- número de trocas ou movimentações realizadas no vetor.

Os experimentos seguem os três problemas definidos no enunciado: comparação entre algoritmos quadráticos em diferentes ordens de entrada, análise do Quick Sort clássico em caso degenerado e avaliação de Insertion Sort e Shell Sort em dados quase ordenados. Os resultados brutos foram gravados em arquivos `.csv` e `.txt` na pasta `data`. Para garantir reprodutibilidade nos vetores aleatórios ou quase ordenados, foi utilizada semente fixa com `srand(42)`.

3 Repositório e Reprodutibilidade

O projeto está disponível publicamente no GitHub, contendo os códigos-fonte, os scripts auxiliares, os dados brutos gerados durante os testes.

- Repositório: <https://github.com/acacio90/benchmarking-sorting-analysis-algorithm>
- Pasta **src**: contém os arquivos-fonte implementados em linguagem C;
- Pasta **data**: contém os arquivos **.csv** e **.txt** com os resultados brutos dos experimentos, além do gráfico gerado para o Problema 2;
- Pasta **scripts**: contém o script auxiliar utilizado para gerar o gráfico de barras do Problema 2;
- Arquivo **README.md**: apresenta a descrição do projeto e os comandos necessários para compilação e execução.

Para compilar o programa principal, foi utilizado o compilador GCC. Um exemplo de comando utilizado é apresentado a seguir:

```
gcc src/main.c src/ordenacao.c -lm -o programa
```

Após a compilação, o programa pode ser executado pelo terminal com o seguinte comando:

```
./programa
```

No Windows, a execução pode ser realizada por:

```
programa.exe
```

Os resultados gerados foram salvos em arquivos na pasta **data**, permitindo a conferência posterior dos tempos de execução, número de comparações e número de trocas ou movimentações. O gráfico do Problema 2 foi gerado a partir dos dados finais utilizando o script presente na pasta **scripts**.

4 Seção dos Testes e Ambiente de Execução

4.1 Ambiente utilizado por Gabriel Pereira do Carmo

Os experimentos foram executados no computador de Gabriel Pereira do Carmo, utilizando sistema operacional Windows e compilador GCC. A compilação e execução dos programas foram realizadas localmente via terminal.

O ambiente utilizado foi:

- Processador: Intel(R) Core(TM) i3-1005G1 CPU @ 1.20GHz;
- Memória RAM: 12,0 GB;
- Tipo de sistema: Sistema operacional de 64 bits, processador baseado em x64;

- Sistema operacional: Windows;
- Ambiente de execução: terminal PowerShell e ambiente Linux via WSL.

A medição do tempo de execução foi realizada por meio da função `clock_gettime`, utilizando `CLOCK_MONOTONIC`. Os dados gerados foram armazenados em arquivos `.csv` e `.txt` na pasta `data`, permitindo a reprodução e análise posterior dos resultados.

4.2 Ambiente utilizado por Pedro Acácio Rodrigues

Os experimentos também foram executados no computador de Pedro Acácio Rodrigues, utilizando sistema operacional Linux e compilador GCC. A compilação e execução dos programas foram realizadas por meio do Windows Subsystem for Linux (WSL), permitindo a utilização de um ambiente compatível com distribuições Linux para desenvolvimento e testes.

O ambiente utilizado foi:

- Processador: Intel Core i7-14700 (20 cores, 28 threads) - 5.30 GHz;
- Memória RAM: 16 GB;
- Tipo de sistema: Sistema operacional de 64 bits, processador baseado em x64;
- Sistema operacional hospedeiro: Windows;
- Ambiente de execução: WSL (Windows Subsystem for Linux);
- Sistema operacional convidado: Linux (Ubuntu).

Assim como nos demais experimentos, a medição do tempo de execução foi realizada por meio da função `clock_gettime`, utilizando `CLOCK_MONOTONIC`. Os resultados foram armazenados em arquivos `.csv` e `.txt`, permitindo a análise e a reprodução dos experimentos.

5 Descrição de Atividades

O código foi organizado em três arquivos principais. O arquivo `main.c` coordena a geração dos vetores, a execução dos experimentos e a gravação dos resultados. O arquivo `ordenacao.c` contém as implementações dos algoritmos de ordenação, enquanto `ordenacao.h` define a estrutura de métricas e os protótipos das funções.

Foram implementados e instrumentados os seguintes algoritmos:

- Insertion Sort;
- Selection Sort;
- Bubble Sort Melhorado;
- Shell Sort;
- Quick Sort clássico, com pivô no último elemento do subvetor;
- Heap Sort.

5.1 Problema 1

No primeiro experimento foram avaliados Insertion Sort, Selection Sort e Bubble Sort Melhorado em vetores com $N = 30000$ elementos. Foram usados três cenários:

- vetor aleatório;
- vetor ordenado em ordem crescente;
- vetor inverso, em ordem decrescente.

5.2 Problema 2

No segundo experimento foram comparados Heap Sort e Quick Sort clássico em vetor ordenado crescente com $N = 5000$. Cada algoritmo foi executado 10 vezes de forma independente. A partir dos tempos obtidos, foram calculados o tempo médio de execução e o desvio padrão.

O Quick Sort utilizado segue a estratégia clássica indicada no enunciado, escolhendo sempre o último elemento do subvetor corrente como pivô.

5.3 Problema 3

No terceiro experimento foram comparados Insertion Sort e Shell Sort em vetor quase ordenado com $N = 50000$. O vetor foi inicialmente criado em ordem crescente e, em seguida, 0,5% dos elementos foram escolhidos para troca com seus vizinhos imediatos.

6 Análise dos Resultados

6.1 Problema 1: Operações Teóricas vs. Tempo Real

Cenário	Algoritmo	Tempo (s)	Comparações	Trocas
Aleatório	Insertion Sort	0.929437	225551463	225521470
Aleatório	Selection Sort	1.542951	449985000	29982
Aleatório	Bubble Sort Melhorado	3.850801	449885077	225521470
Ordenado	Insertion Sort	0.000095	29999	0
Ordenado	Selection Sort	1.316868	449985000	0
Ordenado	Bubble Sort Melhorado	0.000106	29999	0
Inverso	Insertion Sort	1.790499	449985000	449985000
Inverso	Selection Sort	1.606889	449985000	15000
Inverso	Bubble Sort Melhorado	3.146723	449985000	449985000

Tabela 1: Resultados do Problema 1.

Como apresentado na Tabela 1, o Selection Sort manteve exatamente o mesmo número de comparações nos três cenários: 449.985.000. Isso ocorre porque o algoritmo sempre percorre o restante do vetor em busca do menor elemento, independentemente da ordem inicial dos dados.

O Insertion Sort apresentou comportamento fortemente dependente da entrada. No vetor ordenado, realizou apenas 29.999 comparações e nenhuma troca, pois cada elemento já estava em sua posição relativa correta. No vetor inverso, atingiu o pior caso, com 449.985.000 comparações e o mesmo número de movimentações.

No vetor aleatório, o Insertion Sort foi mais rápido que o Selection Sort, mesmo realizando muito mais trocas. Isso indica que o custo real não depende apenas da quantidade de movimentações, mas também da estrutura dos laços e da forma de acesso aos dados. O Bubble Sort Melhorado apresentou o pior tempo no cenário aleatório, pois realizou número elevado de comparações e movimentações.

6.2 Problema 2: Quick Sort Clássico em Vetor Ordenado

Algoritmo	Tempo médio (s)	Desvio padrão (s)
Heap Sort	0.001553	0.000917
Quick Sort	0.066244	0.023737

Tabela 2: Resumo das 10 execuções do Problema 2.

Algoritmo	Comparações	Trocas
Heap Sort	112126	60932
Quick Sort	12497500	0

Tabela 3: Métricas operacionais observadas no Problema 2.

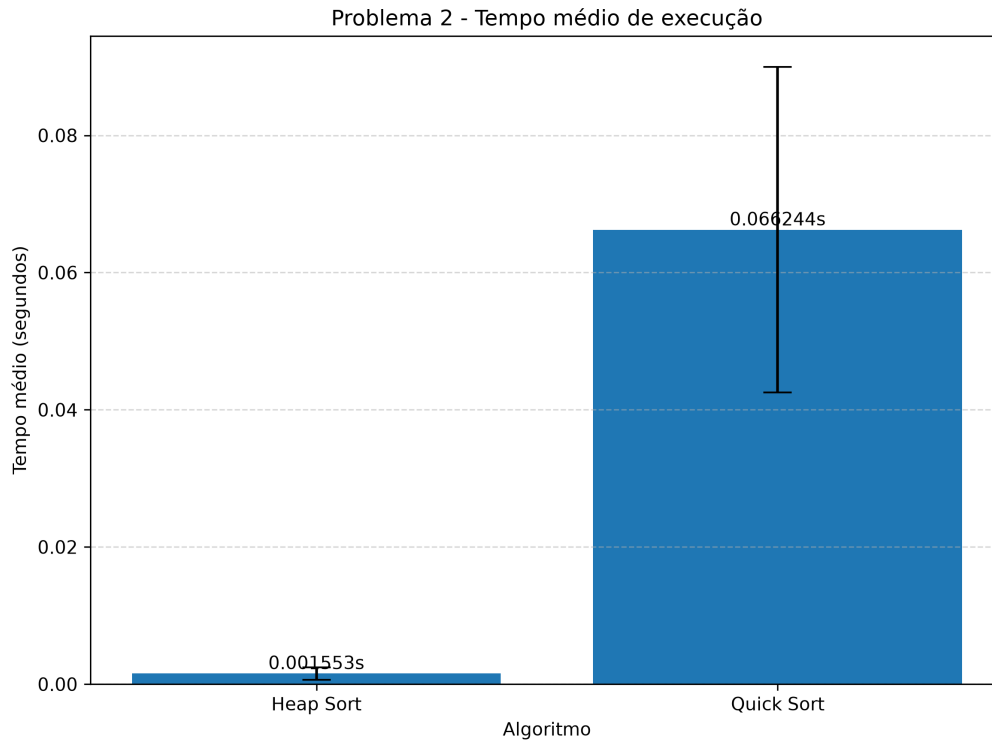


Figura 1: Tempo médio do Heap Sort e Quick Sort com desvio padrão.

Como indicado na Tabela 2 e na Figura 1, o Quick Sort clássico teve desempenho muito inferior ao Heap Sort no vetor já ordenado. Isso ocorre porque, ao escolher sempre o último elemento como pivô, o Quick Sort seleciona o maior elemento do subvetor corrente em cada chamada. Assim, as partições ficam extremamente desbalanceadas: uma partição contém praticamente todos os elementos restantes e a outra fica vazia.

Esse comportamento transforma o Quick Sort em seu caso degenerado, com custo quadrático. Conforme a Tabela 3, para $N = 5000$, foram observadas 12.497.500 comparações. Já o Heap Sort manteve comportamento mais estável, com 112.126 comparações, pois sua estrutura de heap não depende da escolha de pivôs.

6.3 Problema 3: Dados Quase Ordenados e Shell Sort

Algoritmo	Tempo (s)	Comparações	Trocas
Insertion Sort	0.000570	50247	248
Shell Sort	0.004887	700254	248

Tabela 4: Resultados do Problema 3.

De acordo com a Tabela 4, no vetor quase ordenado, o Insertion Sort foi mais eficiente que o Shell Sort. Embora o Shell Sort tenha sido concebido como uma evolução do Insertion Sort ao permitir comparações e movimentações com saltos maiores, esses saltos não trouxeram vantagem neste cenário específico.

Como apenas 0,5% dos elementos foram trocados com vizinhos imediatos, o vetor permaneceu muito próximo da ordenação final. O Insertion Sort se beneficia diretamente

desse tipo de entrada, pois precisa fazer poucas movimentações locais para restaurar a ordem. O Shell Sort, por outro lado, ainda executa várias passagens com diferentes gaps, aumentando o número de comparações mesmo quando o vetor já está quase ordenado.

7 Conclusão

Os experimentos realizados evidenciam que o desempenho dos algoritmos de ordenação depende fortemente das características da entrada. No Problema 1, o Selection Sort manteve número fixo de comparações em todos os cenários, enquanto Insertion Sort e Bubble Sort Melhorado variaram de forma significativa conforme o grau de ordenação do vetor.

No Problema 2, o Quick Sort clássico apresentou desempenho ruim no vetor ordenado devido à escolha do último elemento como pivô. Essa estratégia gerou partições desbalanceadas e levou o algoritmo ao caso degenerado. O Heap Sort, por sua vez, manteve comportamento mais estável e foi consideravelmente mais rápido nesse cenário.

No Problema 3, o Insertion Sort superou o Shell Sort em um vetor quase ordenado. Como as desordens eram locais e pouco numerosas, o Insertion Sort precisou de poucas movimentações para finalizar a ordenação, enquanto o Shell Sort ainda executou múltiplas passagens com diferentes gaps.

Dessa forma, os resultados reforçam que a escolha de um algoritmo de ordenação deve considerar não apenas sua complexidade assintótica, mas também o perfil dos dados de entrada e os custos reais observados durante a execução.